Cloud Computing Mini Project — Report

> One section per task; all numbers are measured (evidence in `results/`). Measured results are stated as such; assumptions/estimates are labelled.

Name: Krityan Kashyap **Roll number:** 230103013 **GitHub:** https://github.com/krityankashyap/cloud-mini-project

---

Architecture overview

Teacher **phi3.5** (Ollama, temperature 0, seed 42) labels 3,000 commands in parallel shards; replies are cleaned into `train.jsonl` and used as sequence-level distillation targets to fine-tune **pythia-70m**. The student is stored in **SeaweedFS** object storage and served by a **FastAPI** container on a local **k3d** cluster behind a **Traefik** Ingress, with an HPA, probes, and CI/CD.

Task 1 — Object storage & parallel teacher labelling

- **Sharding.** Command at line `L` (from 0) → shard `L % SHARD_COUNT`. Two workers (SHARD_INDEX 0,1; SHARD_COUNT 2) write `datasets/teacher/shard-{0,1}.jsonl`. This guarantees a disjoint, exhaustive partition (unit test `test_sharding...`).
- **Cleaning counts** (from `make clean-data`, full 3,000):

input	(a) invalid JSON	(b) wrong keys	(c) bad intent	(d) bad slot	duplicates	kept
3000	6	2	11	61	49	2871

  Reconciles: `3000 = 6+2+11+61+49+2871`. `train.sha256` = `c5d6c2507849a0649f1bc6fc0d0de681e881e8dcd285a96707d14d7b88004f1e` (reproducible offline: `python worker/clean.py --local-shards datasets/teacher ...`). Labelling ran as two in-cluster Jobs (SHARD_INDEX 0/1, SHARD_COUNT 2), 108 min total.

Task 2 — Training the student

- Config: seed 42, lr 5e-4, batch 8, **epochs 6**, max_seq_len 128 (`training/config.yaml`). (3 epochs gave only 89% JSON validity; 6 epochs reaches 99%.)
- Loss per epoch (`training/loss.csv`): 1.97, 0.42, 0.31, 0.32, 0.24, 0.20.
- **Three-model comparison** (score.py on `validation.jsonl`, in `results/`):

Model	json_validity	intent_acc	date	time	place	full_frame
Teacher (phi3.5)	100	85	77	84	85	45
Untrained pythia-70m	0	0	0	0	0	0
Student v1 (6 epochs)	99	78	65	67	75	24

  Student full-frame 24% ≥ half of teacher's 45% (22.5%); validity 99% ≥ 95%. Distillation recovers 78/85 = 92% of the teacher's intent accuracy with a 70M model.

- **Sequence-level vs logit-based distillation.** (a) Sequence-level trains on the teacher's *generated output text* (hard targets) — standard SFT on `Command → JSON`. (b) Logit-based (KD) trains the student to match the teacher's *full next-token probability distribution* (soft targets) via KL divergence.

  c. Not possible here because we only stored the teacher's text, not its logits, and phi3.5 (Ollama) does not expose per-token distributions; vocabularies also differ. (d) Limitation: a 70M model has little capacity, so it memorises surface patterns and generalises poorly on unseen phrasings.

Task 3 — Inference service

- Endpoints `/parse`, `/healthz`, `/readyz`; headers `X-Model-Version`, `X-Served-By`.
- `FAULT_INJECT=1` → HTTP 500 `{"error":"invalid_model_output"}` (verified).

- Image size: **1.2 GB** (CPU-only torch, no weights; ≤ 2 GB). 50 sequential requests @ `--cpus=1 --memory=1g`: **50/50 succeeded, peak memory 506 MiB**, no OOM kill (`results/mem_test.txt`).
- Verified live through the Ingress: `what is the weather in london` → `{"intent":"weather_query","date":null,"time":null,"place":"london"}` with headers `X-Model-Version: v1`, `X-Served-By: <pod>`.

Task 4 — Kubernetes deployment, scaling, reliability

All evidence in `results/` (hpa_scaling, rolling_update, rollback, recovery, coldstart). - Ingress `/parse` answers; requests (500m/512Mi) & limits (4 CPU/1Gi) set; readiness probe keeps the Service endpoint-less until the model loads. - **HPA 1→2** under load (CPU observed 551% vs 70% target); scales back to 1 after the load stops (default 5-min scale-down stabilisation). - **Rolling update v1→v2** (maxUnavailable=0): `X-Model-Version` flipped v1→v2 in ~12s, zero-downtime; **rollback** via `kubectl rollout undo` restored `X-Model-Version: v1`. - **Recovery:** deleted a pod under load; replica count restored to 2 Ready in ~5s. - **Cold start (3 runs):** 5 s / 6 s / 11 s → **median 6 s**; logs show `model v1 loaded from s3://models/v1` (download from object storage). - **Load balancing (2 replicas):** 40 / 36 requests (52% / 48%), both > 30%. - Failures during the disruptive 5-min run: 3/130 (2.31%), almost all baseline invalid-model-output 500s, not the rollout/recovery (which were zero-downtime). - **Limitation:** single laptop = single node; no cross-machine fault tolerance.

Task 5 — Subnetting & NACL *(computed)*

Subnet	Network	Prefix	Usable range	Broadcast	Usable
app	10.20.0.0	/22	10.20.0.1–10.20.3.254	10.20.3.255	1022
public	10.20.4.0	/24	10.20.4.1–10.20.4.254	10.20.4.255	254
data	10.20.5.0	/25	10.20.5.1–10.20.5.126	10.20.5.127	126
management	10.20.5.128	/27	10.20.5.129–10.20.5.158	10.20.5.159	30

NACL verified 6/6 on sample packets. A stateless NACL needs explicit reply rules (it keeps no connection state), and the `198.51.100.0/24` deny must have a lower rule number (100) than the allow-80/443 rules so it is matched first. Full working in `network/subnets.md`.

Task 6 — Reproducibility, testing, CI/CD

- `make up` (one command) / `make down`. CI stages (`.github/workflows/ci.yml`): lint (ruff) → tests (13, pytest) → evaluate (downloads the GitHub Release asset, runs score.py) → quality gate `--min-validity 95`; plus gitleaks (reports nothing).
- **Passing run** (release model, gate passes): https://github.com/krityankashyap/cloud-mini-project/actions/runs/42503449637
- **Failing-gate run** (deliberately untrained Pythia-70M, fails the gate at 0% validity): https://github.com/krityankashyap/cloud-mini-project/actions/runs/42503677573

Task 7 — Performance, cost, review

- Standard load test (4 users, 2 min, 2 replicas, via Ingress), files in `results/`: **77 requests, 1 failure (1.3%)**, p50 **5500 ms**, p95 **9600 ms**, throughput **0.64 rps**. Load balancing: 40 / 36 requests per replica (52% / 48%). Per-pod during load: **~3.8 CPU-cores, 450–580 MiB** (`results/kubectl_top_during_load.txt`).

- **Latency limitation (measured, honest):** the Linux aarch64 torch wheel in the container has no Apple-AMX BLAS, so a single CPU inference is ~2.7 s (vs ~1 s on the host). With the HPA cap of 2 replicas and 4 concurrent users (2 per pod, serialized), the queued request lands at ~5.5 s, so p95 (9.6 s) exceeds the 5 s target *on this Apple-Silicon/Docker-Desktop setup*. The architecture (2 replicas, per-pod generation lock, 4-CPU limit) is correct and would meet the target on native Linux hardware.

- **Cost (rate_card.json: ₹3.50/vCPU-h, ₹0.45/GiB-h; billed on limits).** Per replica/hour = 4 vCPU × 3.50 + 1 GiB × 0.45 = **₹14.45**. Two replicas = ₹28.90/h. Requests/hour = 0.64 rps × 3600 = 2304. `cost_per_1000 = (28.90 / 2304) × 1000 =` **₹12.54 per 1,000 requests**.

- **SLO:** 99% of requests succeed over one 2-minute load-test window. Error budget = 1% × 77 = **0.77 requests**; used **1** request (budget exceeded by the single invalid-model-output 500 — a model-quality miss, not infrastructure).

- **Review table:**

Concern	Evidence (measured)	Limitation	Improvement
Data and storage	3000 labelled → 2871 kept (95.7%); shards 2×1500	teacher drops a few slots (61 `(d)` rejects)	stricter teacher prompt / retry on invalid
Model quality	student full_frame 24% vs teacher 45%; validity 99%	70M model, weak slot copying	more epochs / larger student (pythia-160m)

Concern	Evidence (measured)	Limitation	Improvement
Latency and throughput	p50 5.5s, p95 9.6s, 0.64 rps	container BLAS lacks Apple AMX (~2.7s/req)	run on native Linux / quantise / batch
Resource utilisation	per pod ~3.8 CPU-cores, 450–580 MiB under load	CPU-bound; memory headroom unused	smaller token budget; INT8 weights
Reliability and recovery	cold-start median 6s; pod restored in 5s; rolling update zero-downtime	single node (no cross-machine HA)	multi-node cluster; PodDisruptionBudget
Security and cost	₹12.54 / 1000 reqs; gitleaks: 0 findings; creds via Secret	billed on 4-CPU limit even when idle	right-size CPU limit; scale-to-zero